NavRL Bench

Reinforcement Learning-Based Dynamic Obstacle Avoidance for ROSMASTER M3 Navigation

TECHNICAL REPORT · TRAINING RUN V2_7 · JUNE 2026

TEAM MEMBERS

Boppana Pavan Sai Eshwar Chandra

1010747 · 1010747@mymail.sutd.edu.sg
linkedin.com/in/pavansai18

Alamkaram Teja Srinivasa SivaSaiKumar Raju

1010972 · 1010972@mymail.sutd.edu.sg
linkedin.com/in/sivaalamkaram2002

PROJECT MENTOR

Assoc. Prof. FOONG Shaohui

Engineering Product Development (EPD)
Singapore University of Technology and Design
foongshaohui@sutd.edu.sg

Singapore University of Technology and Design

Engineering Product Development (EPD) · MTD. Robotics and Automation

Code Repository
github.com/pavansai018/navrl-bench
June 2026

ABSTRACT

NavRL Bench presents a reinforcement-learning local controller trained from scratch in IsaacLab / Isaac Sim for the ROSMASTER M3 mecanum-wheel robot. The central contribution is the **ScanHistoryTransformerActor**: a transformer network that processes 8-frame temporal lidar history across 144 rays, engineering 20 per-ray features encoding closing rate, minimum observed range, and directional motion. This enables the policy to infer obstacle dynamics without receiving explicit obstacle state information. The critic receives 33 privileged observations (dynamic obstacle positions, velocities, and collision flags) during training only. PPO via RSL-RL is used with an adaptive curriculum progressing through 15 obstacle difficulty stages followed by 10 domain-randomisation stages. The trained policy is integrated into Nav2 as a drop-in local-controller replacement and benchmarked against DWB, MPPI, and Regulated Pure Pursuit in an AWS warehouse environment under controlled dynamic-obstacle scenarios.

Keywords: reinforcement learning, dynamic obstacle avoidance, transformer policy, PPO, IsaacLab, Nav2, mecanum robot, curriculum learning, domain randomisation, sim-to-real transfer.

Table of Contents

1. Introduction

Classical Nav2 local controllers—DWB, MPPI, and Regulated Pure Pursuit (RPP)—are mature, well-tuned baselines for structured mobile robot navigation. However, they evaluate velocity commands against static occupancy grids updated at sensor frequency, which introduces latency between obstacle appearance and motion adaptation. When fast-moving obstacles repeatedly cross the robot path, these controllers often become overly conservative, stopping frequently or oscillating.

The ROSMASTER M3 is a mecanum-wheel platform capable of simultaneous forward, lateral (strafe), and rotational motion. Classical controllers rarely exploit lateral velocity for obstacle avoidance, leaving significant manoeuvring potential unused. A reinforcement-learning controller trained with a 3D continuous velocity action space can learn lateral bypass manoeuvres that classical baselines cannot produce.

This report describes the complete design and training pipeline for an RL local controller: environment design in IsaacLab, observation and action engineering, a 23-term reward function, an adaptive curriculum with 15 obstacle stages and 10 domain-randomisation stages, PPO training hyperparameters, and Nav2 integration. Section 14 provides a detailed analysis of the current training run (V2_7, 8,022 gradient steps, 7.76 hours), including a critical finding regarding policy exploration saturation.

2. System Overview

The full pipeline proceeds in four stages: (i) RL training in Isaac Sim / IsaacLab with parallel environments, (ii) policy export as a TorchScript checkpoint, (iii) Nav2 integration via a ROS 2 bridge node that assembles observations and publishes `/cmd_vel`, and (iv) benchmark evaluation against three classical Nav2 controllers.

1. **IsaacLab Environment** — ROSMASTER M3 URDF, AWS warehouse map, dynamic cylinder obstacles, 20 Hz control loop.
2. **PPO Training** — RSL-RL, ScanHistoryTransformerActor (policy) + Privileged Critic, adaptive observation/value normalisation.
3. **Adaptive Curriculum** — 15 obstacle difficulty stages → 10 domain-randomisation stages; promotion at 70 % recent success rate.
4. **Policy Export** — TorchScript checkpoint → `nav_rl_bridge` ROS 2 node → Nav2 local controller plugin.
5. **Benchmark** — Controlled trials, same map and scenarios, metrics collected for RL policy and all three classical baselines.

3. Robot Platform and Simulation

3.1 ROSMASTER M3

The ROSMASTER M3 is a compact omnidirectional robot with four independently driven mecanum wheels. Collision radius: 0.22 m. Wheel geometry: radius r = 0.035 m; half-wheelbase along x: 0.0795 m; half-wheelbase along y: 0.09775 m; combined half-wheelbase L = 0.088625 m. Joint names: `lwheel1_Joint`, `lwheel2_Joint`, `rwheel1_Joint`, `rwheel2_Joint`.

3.2 Mecanum Inverse Kinematics

Body-frame velocity (v_x, v_y, ω_z) maps to wheel angular velocities as follows:

$$\omega_{FL} = (v_x - v_y - L\omega_z) / r$$

$$\omega_{RL} = (v_x + v_y - L\omega_z) / r$$

$$\omega_{FR} = (v_x + v_y + L\omega_z) / r$$

$$\omega_{RR} = (v_x - v_y + L\omega_z) / r$$

3.3 Simulation Environment

Simulation runs in NVIDIA Isaac Sim via IsaacLab at 20 Hz (50 ms per control step). The training map is the AWS warehouse environment with pre-recorded Nav2 global-planner paths (up to 600 path points per episode, generated by the MPPI planner on the real `no_roof_warehouse.yaml` map). Dynamic obstacles are cylindrical agents reset at episode boundaries and updated every 30 ms. A map-collision shield prevents any action from driving the robot into static obstacles by zeroing violating velocity components.

4. Observation Space

4.1 Policy (Actor) Observations — 1,176 Dimensions

Table 1. Policy (actor) observation components. Total: 1,176 dimensions.

Component	Dims	Parameters
`local_path_window`	16	8 points, step 8 along global path (x,y relative)
`nav2_heading_error`	1	Signed heading error to next waypoint (rad)
`nav2_cross_track_error`	1	Lateral offset from global path (m)
`scan_history`	1,152	144 rays × 8 frames, max range 10.0 m, step 0.10 m
`base_lin_vel`	2	Body-frame v_x, v_y (m/s)
`base_angle_vel`	1	Body-frame yaw rate ω_z (rad/s)
`previous_action`	3	Last action [v_x, v_y, ω_z]
Total	**1,176**	

> **Design note.**
> The actor does *not* receive `dynamic_obstacles`, `path_blocked`, or `time_to_closest_approach`. Obstacle state is deliberately withheld; the actor must infer motion from temporal scan patterns only. These terms appear in the critic observation space only.

4.2 Critic Observations — 1,209 Dimensions

All 1,176 actor observations plus 33 privileged dimensions:

Table 2. Critic-only privileged observation components (+33 dimensions over actor).

Component	Dims	Content
`dynamic_obstacles`	24	4 obstacles × 6: (x, y, v_x, v_y, radius, active)
`path_blocked`	1	Boolean: obstacle within 0.35 m of any of 32 lookahead points
`time_to_closest_approach`	8	2 obstacles × 4: (ttc, min_dist, rel_vx, rel_vy), horizon 3.0 s
`distance_to_goal`	1	Euclidean distance to final goal (m)
`progress_fraction`	1	Fraction of path completed [0, 1]
`map_collision`	1	Binary: current step map collision
`dynamic_collision`	1	Binary: current step dynamic collision
Critic total	**1,209**	

5. Action Space and Processing Pipeline

5.1 Continuous 3D Velocity Action

The policy outputs a 3-dimensional continuous action in the robot body frame. The network output is in $[-1, +1]^3$ and is then scaled to physical velocity units.

Table 3. Action dimensions, physical bounds, and rate limits per control step (50 ms).

Dimension	Physical Range	Max Δ per Step
v_x — forward velocity	±0.50 m/s	0.025 m/s
v_y — lateral (strafe) velocity	±0.50 m/s	0.025 m/s
ω_z — yaw angular velocity	±1.50 rad/s	0.080 rad/s

5.2 Action Processing Pipeline

Each raw network output passes through seven sequential stages before wheel commands are issued:

1. **Clip to [−1, +1]** — enforce network output bounds.
2. **Scale to physical units** — multiply by [0.5, 0.5, 1.5].
3. **Action delay buffer** — configurable 0–3 step delay (domain-randomised).
4. **Motor strength scaling** — random per-wheel scale factor ∈ [0.85, 1.15] (domain-randomised).
5. **Rate limiting** — clamp change per step to max_delta values above.
6. **Map-collision shield** — zero velocity components that would penetrate static obstacles.
7. **Mecanum inverse kinematics** — convert (v_x, v_y, ω_z) to four wheel ω values.

6. Policy Architecture: ScanHistoryTransformerActor

The actor network processes the 1,176-dimensional observation vector through a transformer-based scan encoder combined with dedicated path and motion encoders. Figure 1 shows the complete data flow.

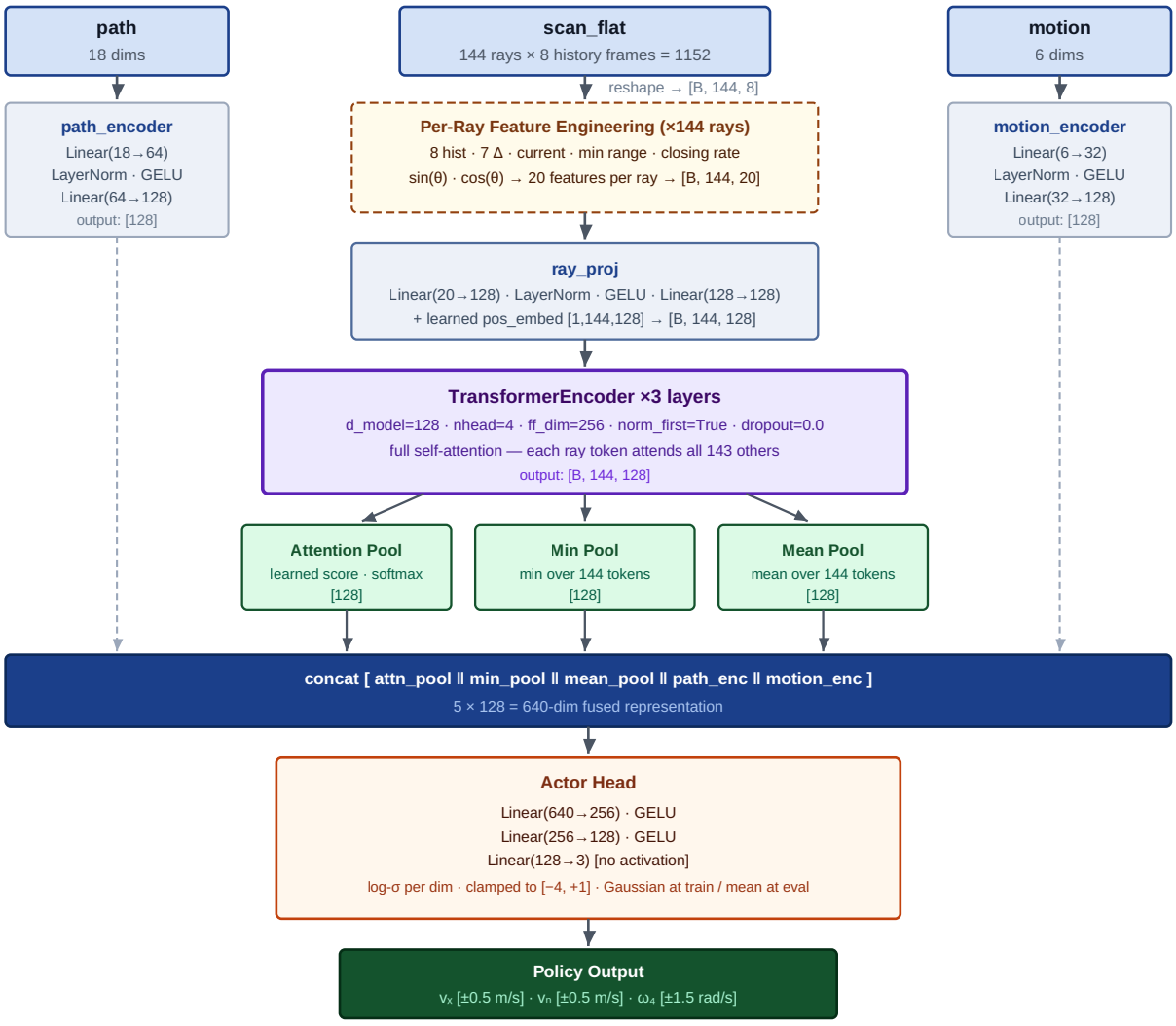

Figure 1. ScanHistoryTransformerActor architecture. Inputs (path 18, scan_flat 1152, motion 6) flow through dedicated encoders; the scan branch uses per-ray feature engineering, a projection layer, a 3-layer transformer encoder, and three-way pooling before concatenation and the actor head.

6.1 Per-Ray Feature Engineering

The 1,152-dimensional `scan_history` tensor is reshaped to [B, 144 rays, 8 frames]. Each ray's 8-frame window is expanded to a 20-dimensional feature vector:

- **8 raw history values** — lidar range at each of the 8 time steps (oldest to newest).
- **7 temporal deltas** — first-order differences $\Delta_t = h_t - h_{t-1}$, capturing closing/opening rate per ray.
- **Current range** — most recent reading (frame 8).

- **Minimum range** — minimum over the 8-frame window (closest approach).
- **Closing rate** — signed rate of range decrease (negative = approaching).
- **sin(θ_{ray})** and **cos(θ_{ray})** — angular position encoding in robot body frame.

Result: input tensor of shape [B, 144, 20].

6.2 Ray Projection Layer

```
ray_proj      = Linear(20 → 128) → LayerNorm(128) → GELU → Linear(128 → 128)
pos_embed     = Embedding(144, 128)   # learned per-ray positional embedding
token[b,r]    = ray_proj(feature[b,r]) + pos_embed(r)
# Shape: [B, 144, 128]
```

6.3 Transformer Encoder

Table 4. Transformer encoder configuration.

Parameter	Value	Notes
d_model	128	Token embedding dimension
nhead	4	Attention heads (head dim = 32)
dim_feedforward	256	Inner FFN dimension
num_layers	3	Stacked encoder blocks
norm_first	True	Pre-LN formulation for stable gradients
activation	GELU	
dropout	0.0	No dropout during training
Input / output shape	[B, 144, 128]	All 144 ray tokens, global self-attention

Each encoder layer applies: LayerNorm → Multi-Head Self-Attention → residual → LayerNorm → FFN($128 \to 256 \to 128$) → residual. The Pre-LN formulation improves gradient flow through 3 layers compared to the original post-LN transformer.

6.4 Three-Way Pooling

After the encoder produces [B, 144, 128], three complementary global representations are computed:

- **Attention pooling** — learned query vector $\in \mathrm{R}^{128}$ attends all 144 tokens; output [B, 128]. Focuses on task-relevant rays.
- **Min pooling** — element-wise minimum across the 144-token sequence; output [B, 128]. Captures the nearest obstacle per feature dimension.
- **Mean pooling** — arithmetic mean across tokens; output [B, 128]. Global scene occupancy summary.

6.5 Path and Motion Encoders

```
path_encoder   = Linear(18 → 64) → LayerNorm → GELU → Linear(64 → 128)
motion_encoder = Linear(6  → 32) → LayerNorm → GELU → Linear(32 → 128)
```

6.6 Concatenation and Actor Head

```
fused = concat([attn_pool, min_pool, mean_pool, path_embed, motion_embed])
      # [B, 5 × 128] = [B, 640]

actor_head:
  Linear(640 → 256) → GELU
  Linear(256 → 128) → GELU
  Linear(128 → 3)          # distribution mean, no output activation

log_std:  learned parameter vector [3], clamped to [−4, +1]
          → σ ∈ [e^−4, e^+1] ≈ [0.018, 2.718]
```

7. Critic Architecture

The critic shares the identical ScanHistoryTransformerActor backbone (per-ray feature engineering, transformer encoder, three-way pooling, path encoder, motion encoder) to process the 1,176-dimensional shared observations. A separate privileged encoder processes the 33-dimensional privileged input:

```
priv_encoder = Linear(33 → 128) → LayerNorm → GELU → Linear(128 → 128)

critic_input = concat([attn_pool, min_pool, mean_pool, path_embed, motion_embed,
priv_embed])
             # [B, 640 + 128] = [B, 768]

critic_head:
  Linear(768 → 256) → GELU
  Linear(256 → 128) → GELU
  Linear(128 → 1)          # scalar value estimate V(s)
```

The critic has access to ground-truth dynamic obstacle state during training only. At inference time, only the actor network is used; the critic is discarded. This asymmetric information structure (privileged critic / blind actor) is a standard technique in RL for complex environments where state information is partially observable at deployment time.

8. Reward Function

Twenty-three reward terms are active during training. The `mppi_imitation` term (weight +25.0) is defined in configuration but disabled. Terms are grouped into five functional categories below.

8.1 Path Following

Table 5. Path following reward terms.

Term	Weight	Key Parameters
`progress`	+35.0	max_step_progress = 0.05 m
`goal_approach`	+5.0	max_step_progress = 0.08 m
`cross_track`	−4.0	max_error = 1.0 m
`path_rejoin`	+15.0	active_threshold = 0.2 m
`heading_alignment`	+8.0	lookahead_index_offset = 4
`path_velocity`	+6.0	Progress-weighted forward speed
`lateral_bypass`	+5.0	max_cte = 0.8 m

8.2 Dynamic Obstacle Avoidance

Table 6. Dynamic obstacle avoidance reward terms.

Term	Weight	Key Parameters
`dynamic_collision`	−100.0	robot_radius = 0.22 m
`dynamic_clearance`	−20.0	clearance = 0.25 m beyond robot edge
`dynamic_ttc`	−15.0	horizon = 3.0 s
`wall_aware_dynamic_bypass`	+20.0	lookahead = 2.0 m, corridor half-width = 0.55 m
`dynamic_yield_no_side_space`	+8.0	Same corridor parameters
`dynamic_forward_blocked`	−12.0	Penalises lingering when blocked; lookahead = 2.0 m
`dynamic_bad_lateral`	−8.0	Penalises choosing wrong bypass side; min_side_clearance = 0.45 m

8.3 Static / Map Safety

Table 7. Static obstacle safety reward terms.

Term	Weight	Key Parameters
`map_collision`	−150.0	robot_radius = 0.22 m (highest penalty)
`static_velocity_clearance`	−15.0	safe_distance = 0.30 m, 144 rays, sector half-angle = π/4 rad

8.4 Goal Completion and Speed Incentives

Table 8. Goal completion and speed incentive terms.

Term	Weight	Notes
`final_goal`	+120.0	Lump-sum on reaching goal within 0.30 m
`start_speed`	−8.0	Penalises slow departure; warmup period = 1.5 s
`no_wait`	−2.0	Penalises speed below 0.10 m/s at any time

8.5 Regularisation

Table 9. Regularisation reward terms.

Term	Weight	Notes
`action_smoothness`	−0.06	Penalises L2 norm of action jerk
`yaw_rate`	−0.05	Penalises excessive rotation
`lateral_oscillation`	−0.35	Penalises rapid lateral velocity sign changes
`time`	−0.06	Per-step time penalty to encourage speed

9. Curriculum Design

9.1 Obstacle Difficulty Curriculum — 15 Stages

Training begins with stationary tiny obstacles placed on the robot path and progresses through increasingly fast and geometrically complex dynamic scenarios. A stage is promoted when the recent success rate exceeds 70 % over a trailing window.

Table 10. Obstacle curriculum stages in order of increasing difficulty.

#	Stage Identifier	Category
1	`center_stationary_tiny`	Static
2	`side_stationary_tiny`	Static
3	`center_stationary_small`	Static
4	`side_stationary_small`	Static
5	`center_stationary_medium`	Static
6	`slow_crossing_far`	Dynamic — slow transverse crossing (far)
7	`slow_crossing_near`	Dynamic — slow transverse crossing (near)
8	`medium_crossing_far`	Dynamic — medium speed crossing (far)
9	`medium_crossing_near`	Dynamic — medium speed crossing (near)
10	`fast_crossing_far`	Dynamic — fast transverse crossing
11	`same_lane_slow`	Dynamic — same direction slow
12	`same_lane_medium`	Dynamic — same direction medium
13	`reverse_same_lane_slow`	Dynamic — head-on approach slow
14	`center_stationary_large`	Static — large obstacle on path
15	`two_crossing_combo`	Dynamic — two simultaneous crossing obstacles

9.2 Domain Randomisation Curriculum — 10 Stages

After all 15 obstacle stages are completed, 10 successive domain-randomisation parameters are activated to improve sim-to-real transfer robustness.

Table 11. Domain randomisation curriculum stages in activation order.

#	Stage Identifier	Effect
1	`lidar_rays`	Random ray angular perturbations
2	`scan_noise`	Gaussian noise on range measurements
3	`scan_dropout`	Structured sector blanking
4	`action_delay`	Random 0–3 step actuation delay
5	`motor_strength`	Per-wheel torque scale $\in$ [0.85, 1.15]
6	`mass`	Robot mass variation
7	`com_shift`	Centre-of-mass offset perturbation
8	`wheel_radius`	Per-wheel radius variation
9	`wheel_slip`	Lateral and longitudinal wheel slip
10	`combined_strong`	All above randomisations at maximum intensity simultaneously

10. PPO Training Configuration

Training uses the RSL-RL PPO implementation.

Table 12. PPO hyperparameters (RSL-RL implementation).

Parameter	Value	Notes
Discount factor γ	0.99	Long time horizon
GAE λ	0.95	Generalised advantage estimation
PPO clip ratio ε	0.2	Standard clipping range
Target KL divergence	0.01	Triggers adaptive LR adjustment
Entropy coefficient	0.00	No entropy regularisation bonus
Log-std clamp range	[−4, +1]	$\sigma \in [0.018, 2.718]$; upper clamp is the critical saturation point
Value function loss	MSE	
Optimiser	Adam	
Observation normalisation	Enabled	Running mean/variance normaliser
Value normalisation	Enabled	EMA-based return normaliser

11. Termination Conditions

Table 13. Episode termination conditions and parameters.

Condition	Parameters	Type
`final_goal_reached`	threshold = 0.30 m	Success — positive termination
`map_collision`	robot_radius = 0.22 m	Failure — static collision
`dynamic_collision`	robot_radius = 0.22 m	Failure — dynamic collision
`stuck`	speed_threshold = 0.02 m/s, window = 2.0 s, grace = 2.0 s	Failure — robot stopped
Time limit	Max episode steps	Timeout — neutral

12. Nav2 Integration

The trained actor is exported as a TorchScript checkpoint and deployed via the `nav_rl_bridge` ROS 2 package as a drop-in Nav2 local controller plugin. The global planner (Navfn/A*), costmaps, and behaviour trees remain unchanged.

12.1 Bridge Architecture

The bridge node subscribes to the Nav2 global path and `/scan` topics, maintains an 8-frame scan history buffer at 20 Hz, extracts 8 lookahead waypoints from the global path, and computes heading and

cross-track errors. The assembled observation vector is passed to the TorchScript actor; the output [v_x, v_y, ω_z] is published on `/cmd_vel`.

12.2 Path Dataset

The training path dataset was recorded from the Nav2 MPPI planner running on the real AWS warehouse map (`no_roof_warehouse.yaml`). This ensures the path distribution seen during training closely matches deployment paths, reducing distribution shift. Up to 600 path points per episode are loaded, reset at episode start.

13. Benchmark Design

13.1 Controllers

- **DWB** — Dynamic Window Approach; classical Nav2 baseline using configurable velocity scoring critics.
- **MPPI** — Model Predictive Path Integral; sampling-based controller evaluating thousands of candidate trajectories.
- **Regulated Pure Pursuit (RPP)** — path-tracking with speed regulation based on curvature and obstacle proximity.
- **PPO + ScanHistoryTransformerActor** — the learned RL policy (this work); outputs 3D mecanum velocity directly.

13.2 Scenarios and Metrics

Table 14. Benchmark test scenarios.

#	Scenario	Key Challenge
1	Fast crossing obstacle	Reaction speed; bypass vs. stop decision
2	Temporary path blockage	Yield vs. circumnavigate decision
3	Goal reaching, multiple movers	Simultaneous avoidance while progressing
4	Static obstacle only (baseline)	Normal navigation, no dynamic elements
5	Narrow passage	Precise lateral positioning; mecanum advantage
6	Repeated controlled trials	Statistical validity; N runs per scenario per controller

Metrics collected: success rate, time to goal, path length, number of stops, collision count, minimum dynamic obstacle clearance, velocity smoothness (jerk norm).

14. Training Progress Analysis

Training run **V2_7** was launched on 2026-06-25 at 15:44:35 and had completed **8,022 gradient steps** over **7.76 hours** at the time of this analysis. The following subsections examine curriculum progress, learning dynamics, episode termination distribution, per-reward contributions, and directional collision analysis.

14.1 Curriculum Progress

The four primary curriculum metrics — success rate, timeout rate, dynamic collision rate, and static collision rate — are shown below. All are computed over a recent trailing window of episodes.

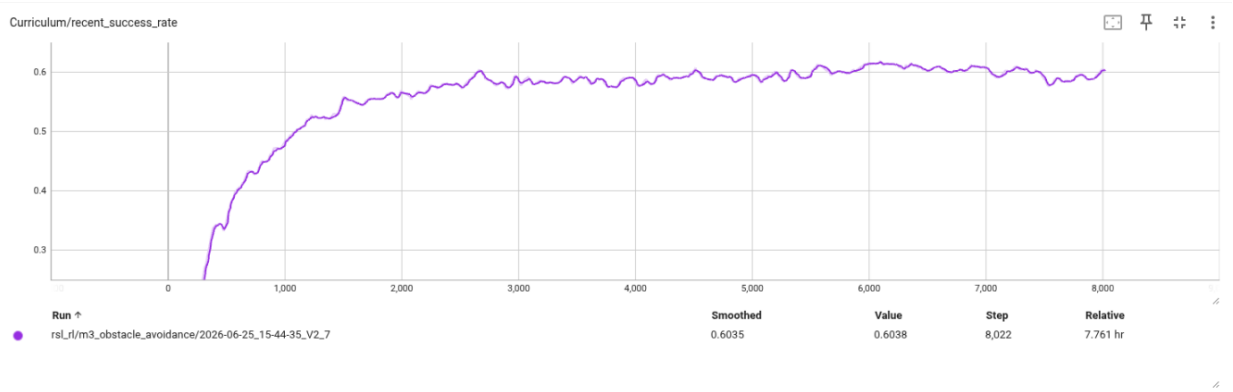

Figure 1. Recent episode success rate (%). The promotion threshold is 70%. The curve plateaued at approximately 60.4%, below the threshold required for advancement to the next curriculum stage.

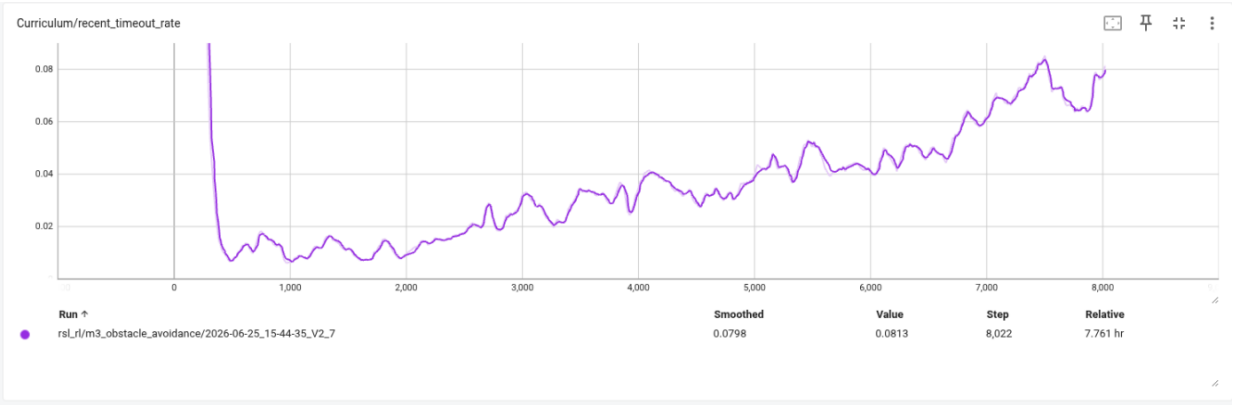

Figure 2. Recent episode timeout rate (%). Episodes that exhausted the maximum step count without reaching the goal or colliding.

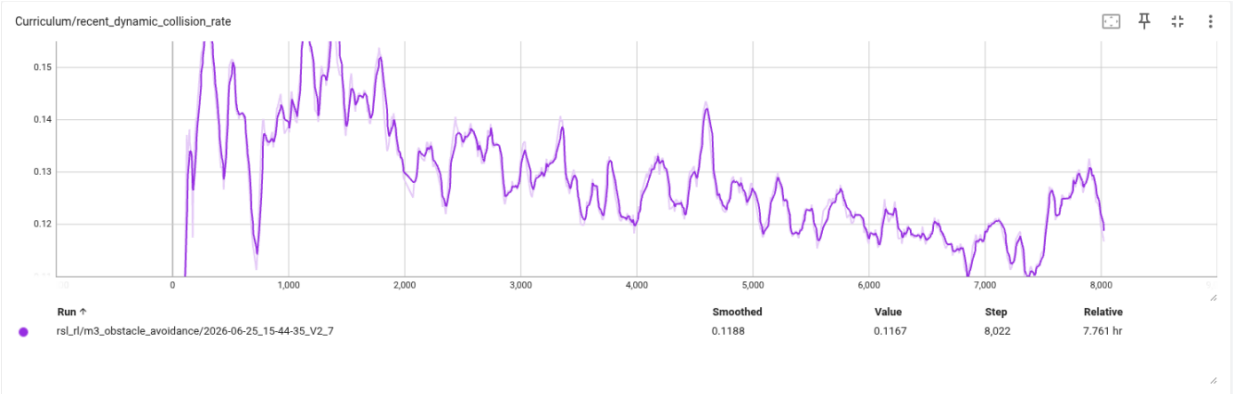

Figure 3. Recent dynamic collision rate (%). Contact events with moving cylindrical obstacles. The declining trend indicates improving but incomplete dynamic avoidance behaviour.

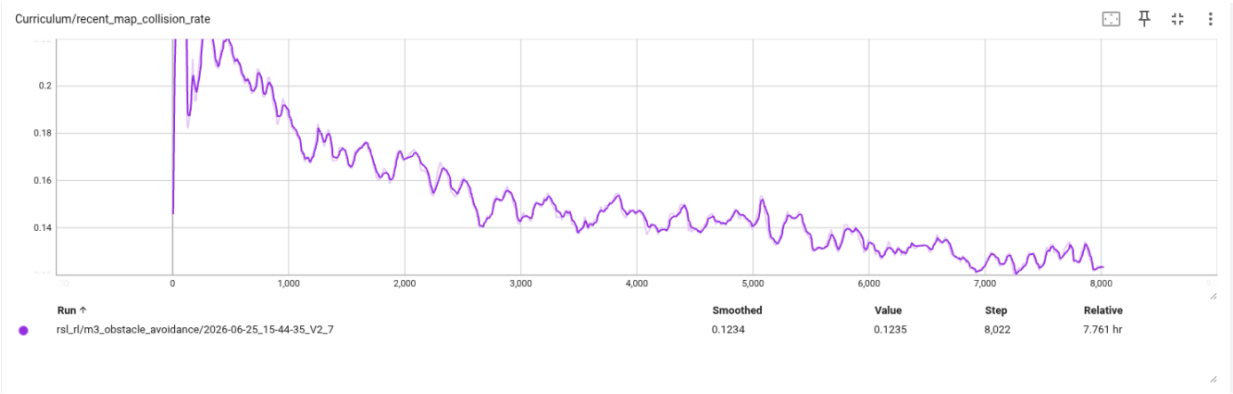

Figure 4. Recent static (map) collision rate (%). Contact events with the static occupancy map.

> **Observation.**
>
> The success rate plateau at ~60.4% indicates the policy has partially solved the current curriculum stage but cannot consistently achieve the 70% threshold. Dynamic collision rate shows a declining trend, suggesting continued policy improvement. However, a separate factor (policy std saturation, Section 14.3) may be limiting further progress.

14.2 Global Training Signals

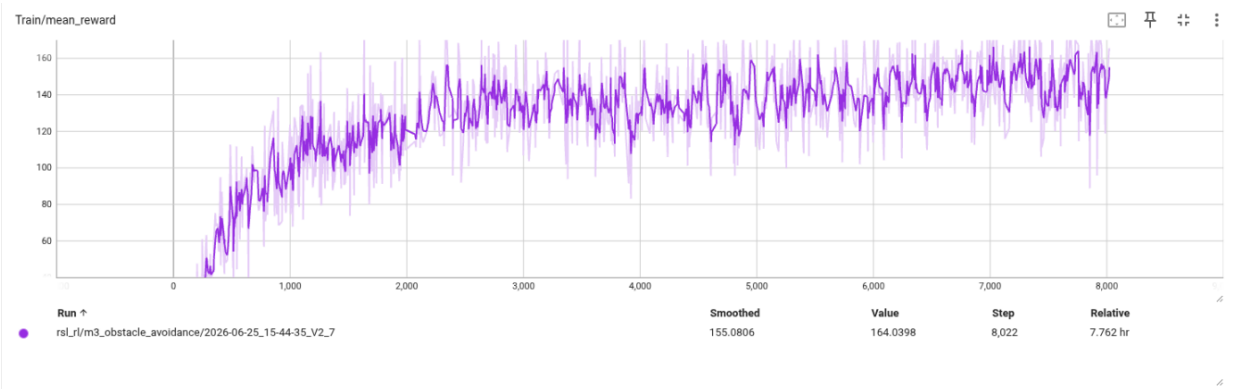

Figure 5. Mean total episodic reward over training steps. The upward trend across 8,022 steps confirms continued policy improvement despite the success-rate plateau.

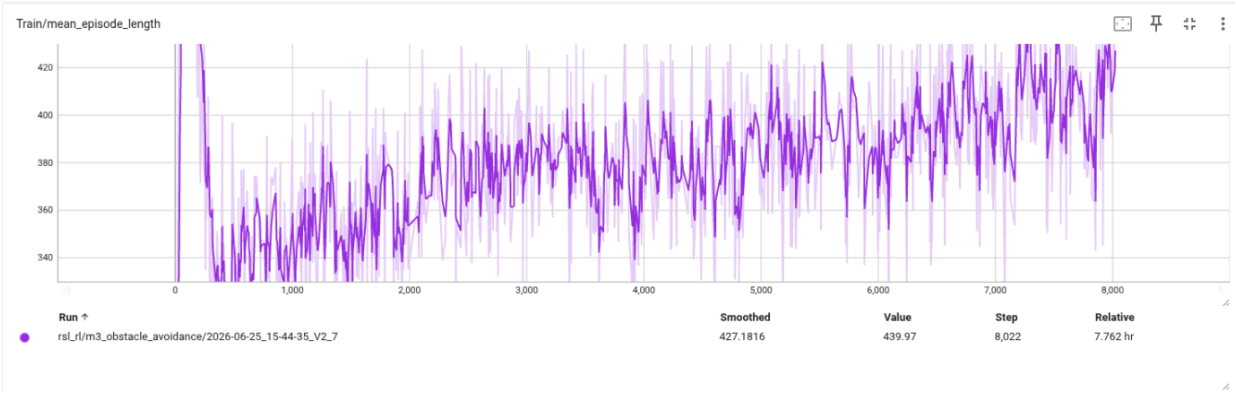

Figure 6. Mean episode length (steps). Longer episodes indicate the robot is surviving longer before termination, consistent with decreasing collision rates.

14.3 Policy Exploration: Standard Deviation and Entropy

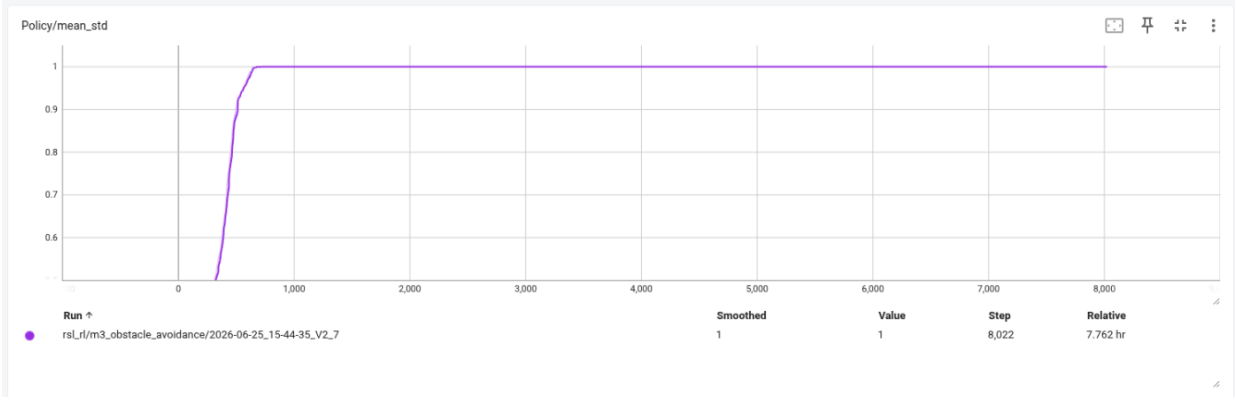

Figure 7. Policy standard deviation averaged over the 3 action dimensions. The value saturated at σ = 1.0 from approximately step 600 and has remained clamped at that value throughout training.

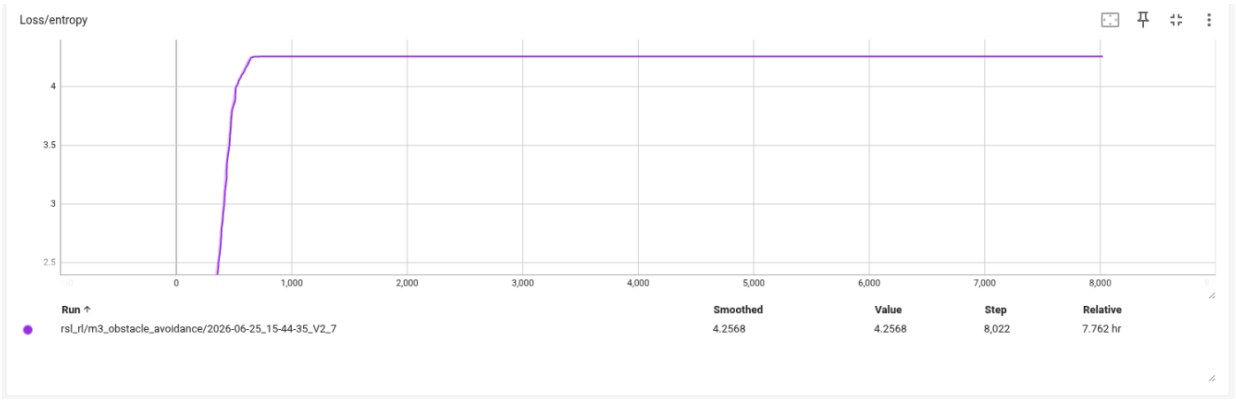

Figure 8. Policy entropy (nats). Flat at H ≈ 4.257 nats throughout training, consistent with the theoretical entropy of a 3-dimensional isotropic Gaussian with σ = 1.0 on all dimensions: $H = 0.5 \times 3 \times (1 + \ln 2\pi) \approx 4.257$ nats.

> **Critical finding — Policy standard deviation saturation.**
>
> RSL-RL clamps the log-std parameter to the range [−4, +1], corresponding to σ ∈ [0.018, 2.718]. The policy mean std has been clamped at the upper bound σ = 1.0 (log-std = 0) since step 600. With no entropy coefficient (entropy_coeff = 0.00), there is no gradient pressure to reduce σ, and the policy remains in a high-entropy regime. At σ = 1.0 with max v_x = 0.5 m/s, approximately 32 % of sampled actions are clipped by the action bounds, introducing noise in executed trajectories. This likely contributes to the success-rate plateau.
>
> **Recommended fix:**
>
> Lower the RSL-RL log-std upper clamp to −0.5 (giving σ ≤ 0.61), or introduce a small entropy penalty (e.g., entropy_coeff = 0.005) to encourage std convergence.

14.4 Episode Termination Distribution

Figures below show how episodes terminate across training. A healthy policy should show increasing *final_goal_reached* and decreasing collision/stuck rates.

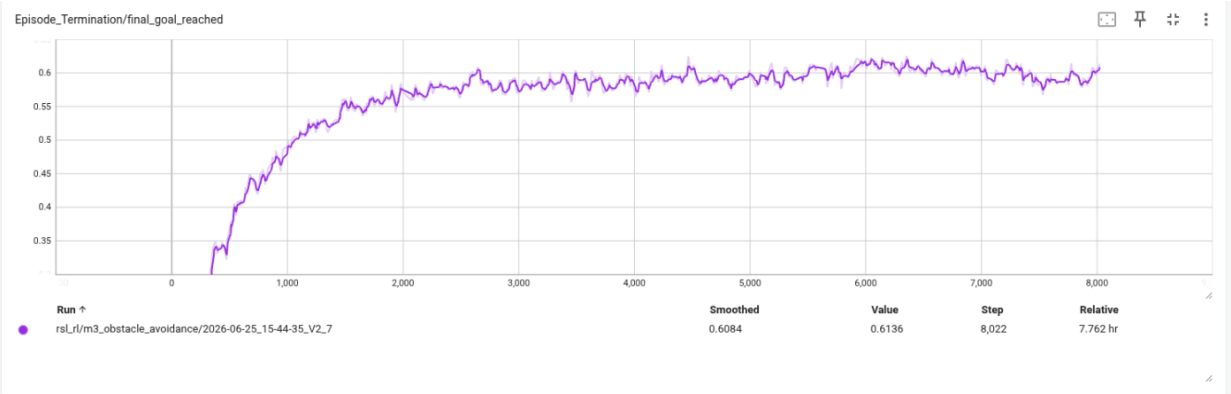

Figure 9. Rate of goal-reached terminations over training steps. This is the primary success signal; it should approach and exceed the 70% curriculum threshold.

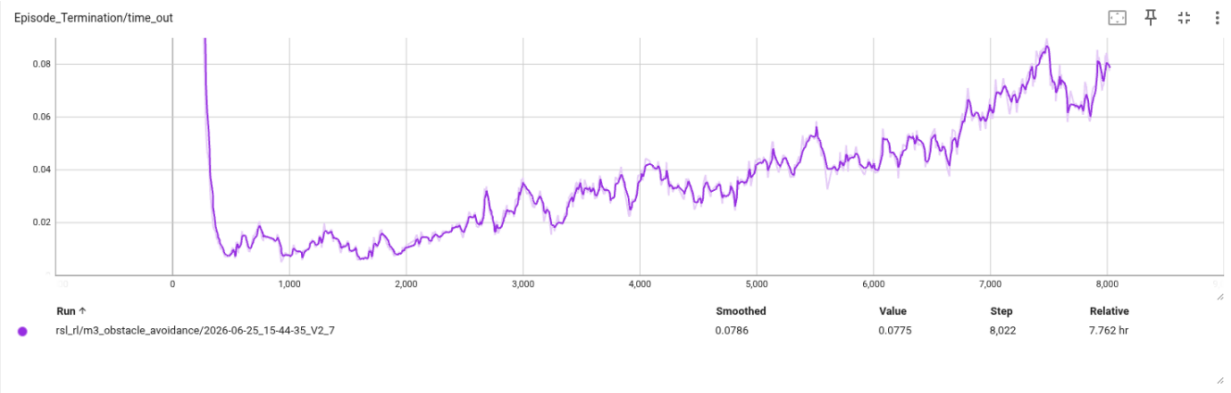

Figure 10. Rate of timeout terminations. Episodes that ran to the step limit without success or failure. Persistent timeouts may indicate path following incomplete on longer routes.

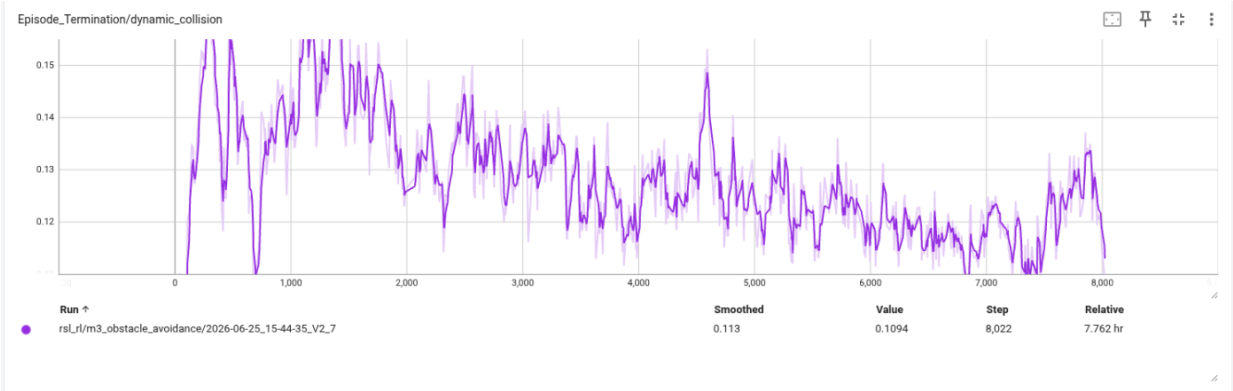

Figure 11. Rate of dynamic collision terminations. Declining trend confirms the policy is improving dynamic obstacle avoidance.

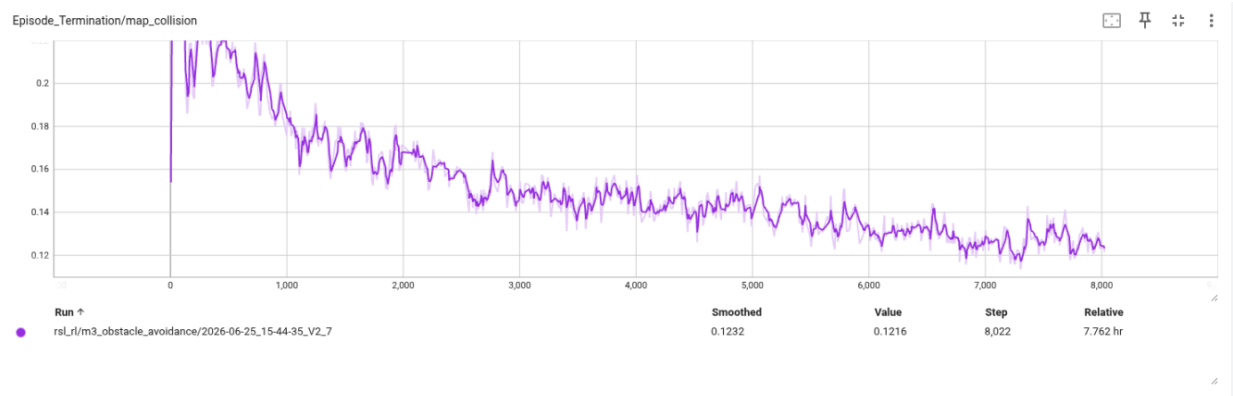

Figure 12. Rate of static (map) collision terminations. Should trend toward zero as the policy learns to avoid walls during bypass manoeuvres.

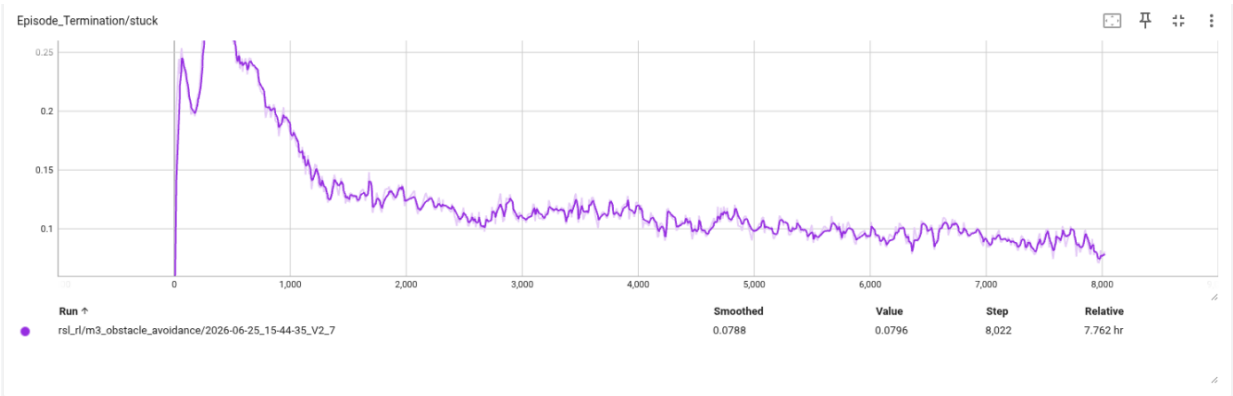

Figure 13. Rate of stuck terminations (speed below 0.02 m/s for more than 2 s after a 2 s grace period). Indicates the robot freezing in front of obstacles instead of choosing a bypass action.

14.5 Per-Reward Component Breakdown

Individual reward component contributions per episode reveal which objectives are being optimised and which remain underperforming.

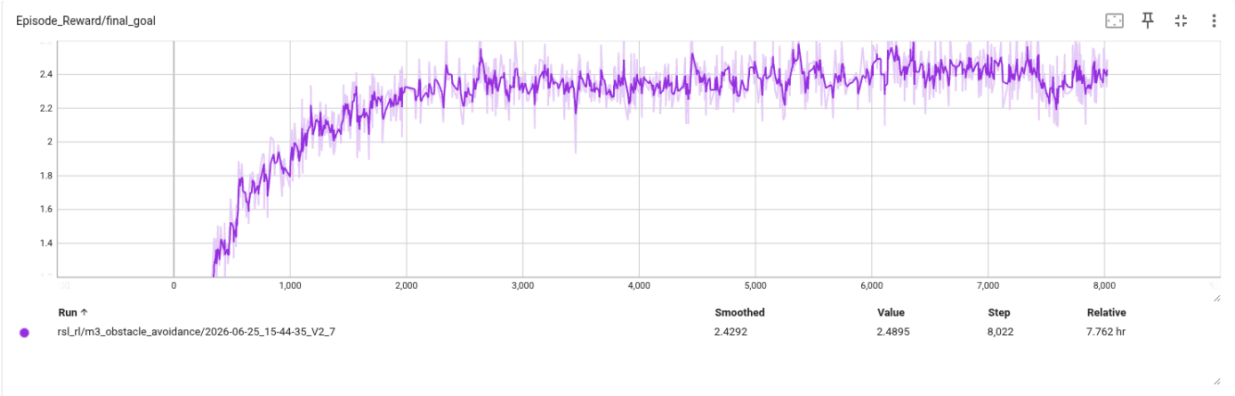

Figure 14. Final goal reward contribution (+120 per success). Tracks the trajectory of success events and their frequency over training.

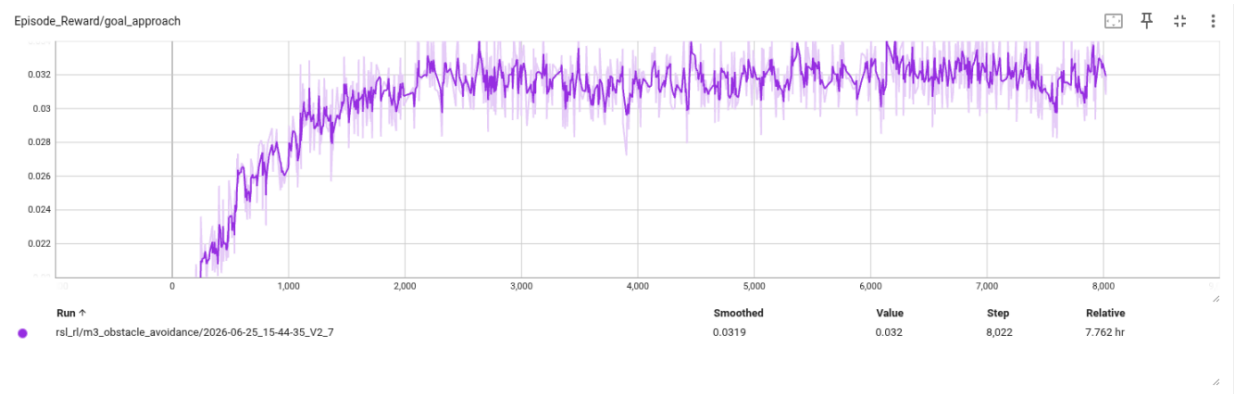

Figure 15. Goal approach reward contribution (+5 weight). Continuous shaping reward for reducing distance to goal each step.

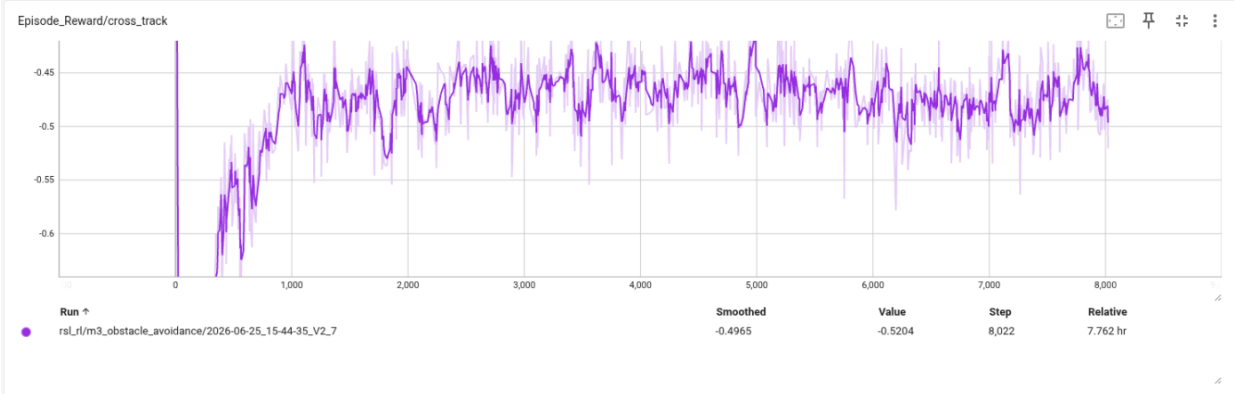

Figure 16. Cross-track error penalty contribution (−4 weight). Measures lateral deviation from the global path. Lower absolute values indicate better path following.

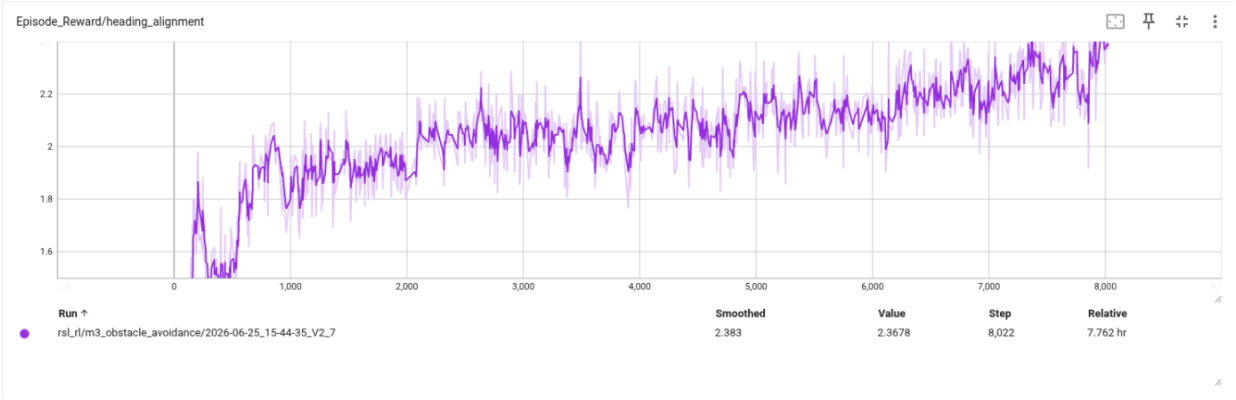

Figure 17. Heading alignment reward contribution (+8 weight). Rewards alignment of robot heading with the global path tangent at lookahead index +4.

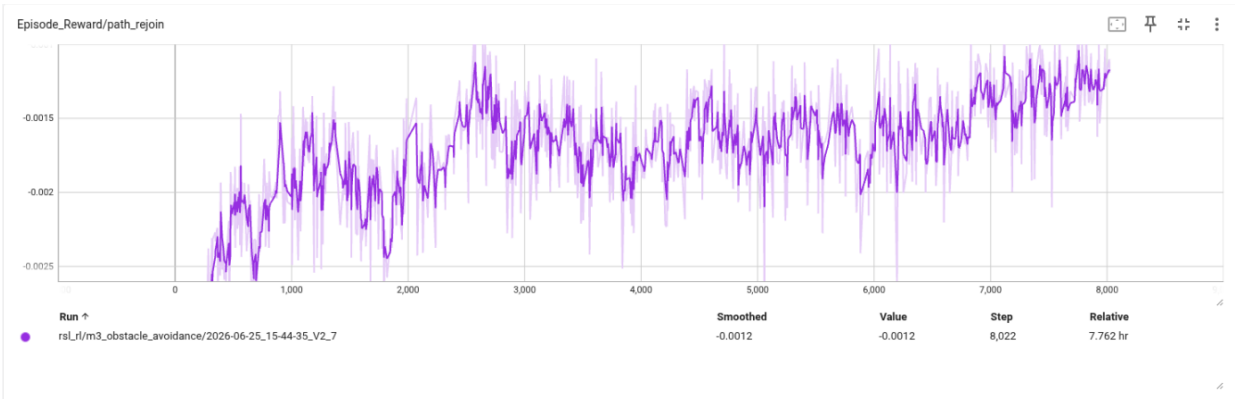

Figure 18. Path rejoin reward contribution (+15 weight). Activated when the robot returns within 0.2 m of the global path after lateral deviation.

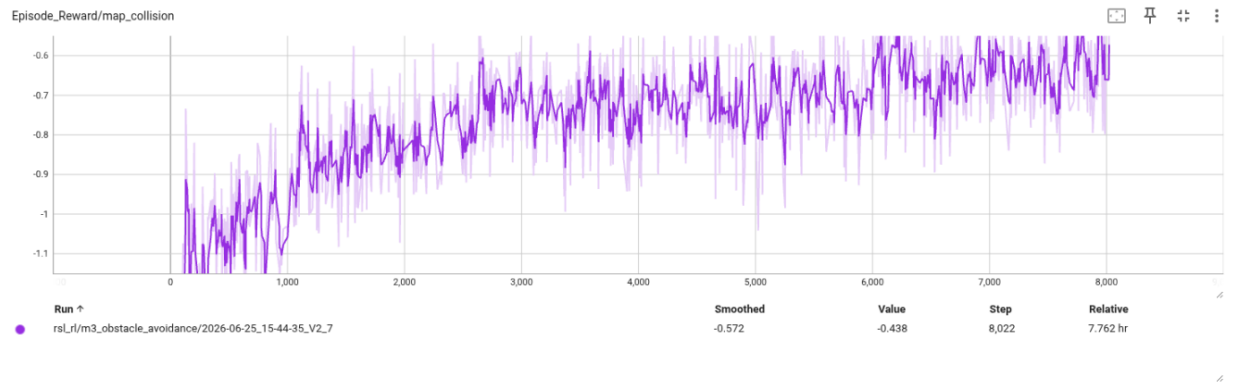

Figure 19. Static collision reward contribution. Represents weighted map-collision events (−150 weight). Should trend toward zero as training progresses.

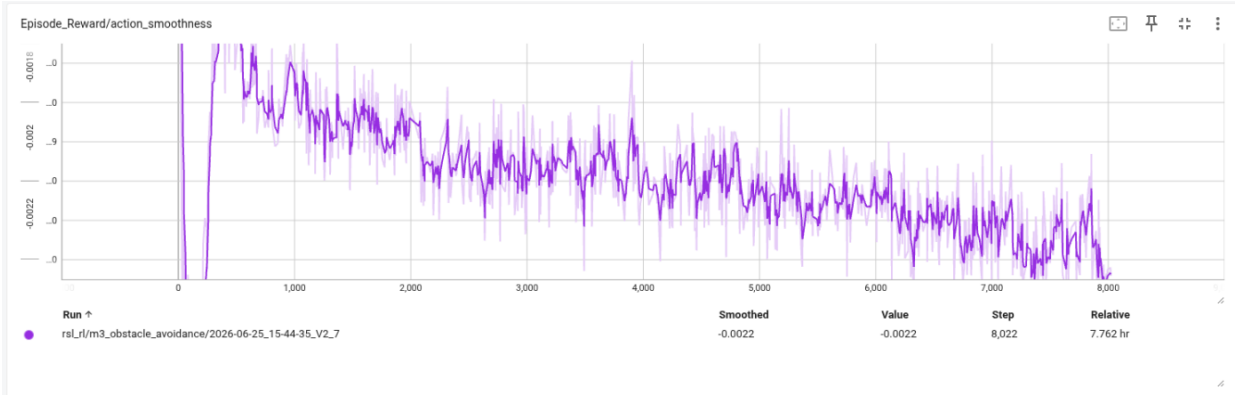

Figure 20. Action smoothness penalty contribution (−0.06 weight). Penalises high-frequency action changes (jerk). Lower absolute values indicate smoother control.

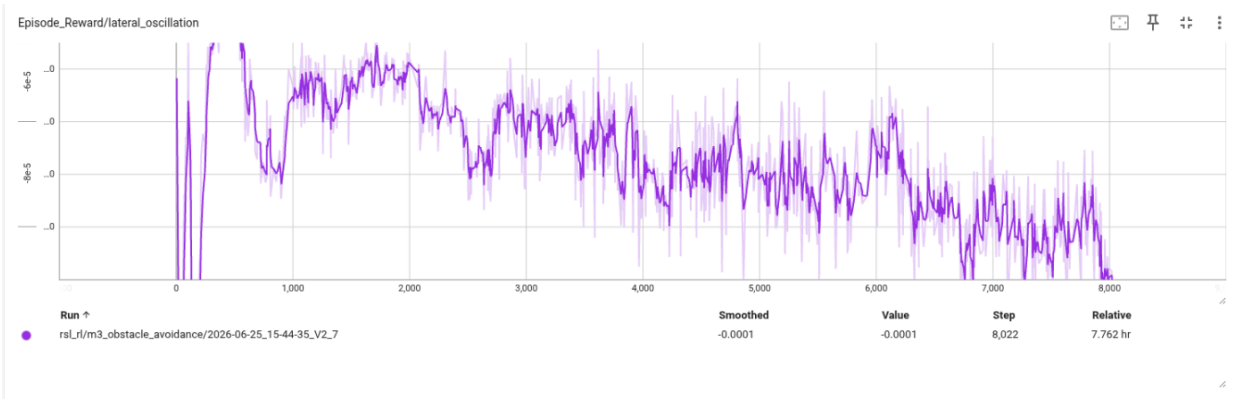

Figure 21. Lateral oscillation penalty contribution (−0.35 weight). Penalises rapid sign changes in lateral velocity, discouraging side-to-side weaving.

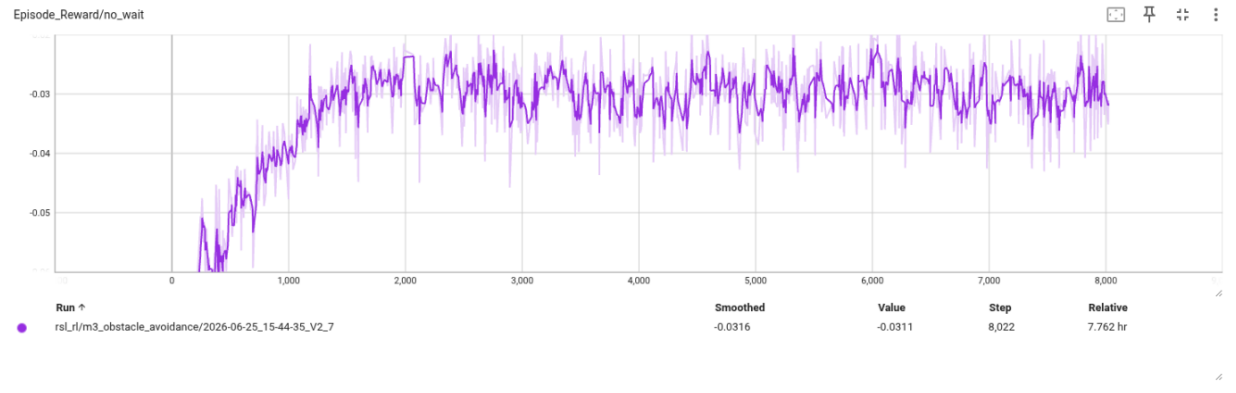

Figure 22. No-wait penalty contribution (−2.0 weight). Penalises robot speed below 0.10 m/s. Sustained high absolute values indicate the robot is frequently stopping.

14.6 Static Collision Directional Analysis

Static collisions are decomposed by direction in the robot body frame. This identifies which navigation manoeuvres most frequently result in wall contact, informing reward tuning or curriculum adjustments.

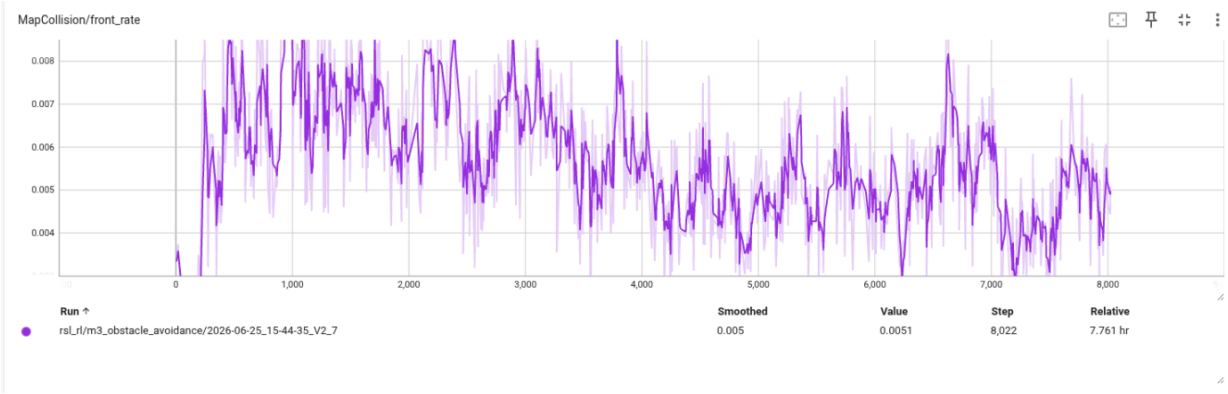

Figure 23. Static (map) collision rate from the front of the robot body. High values indicate forward path planning or speed regulation failures near walls.

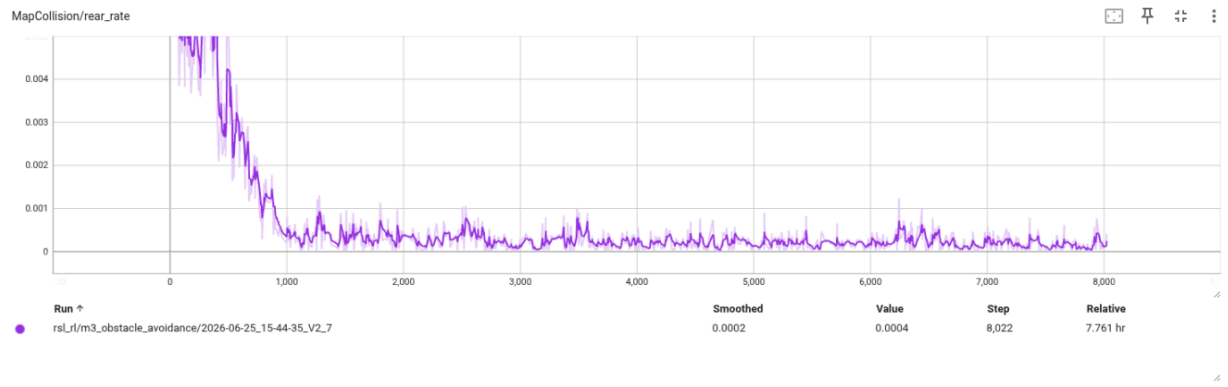

Figure 24. Static collision rate from the rear. Indicates the robot backing into obstacles during yield or reverse manoeuvres.

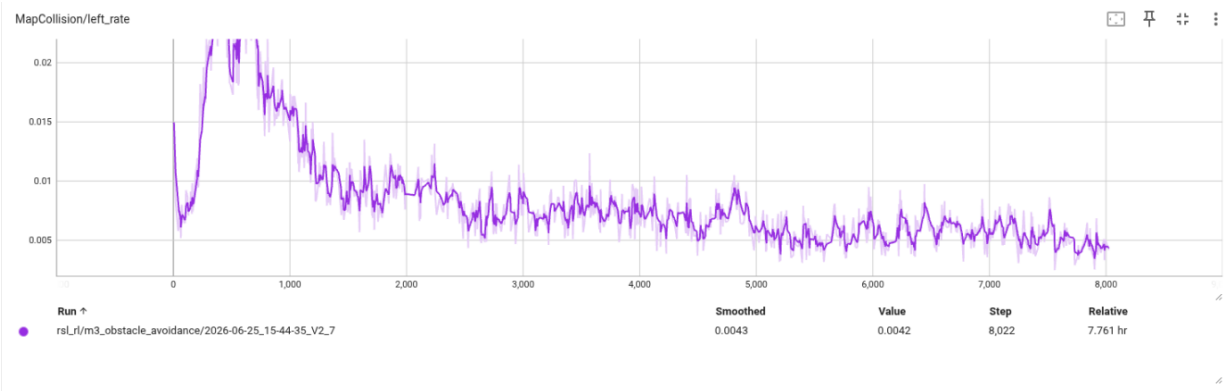

Figure 25. Static collision rate from the left side. Relevant to left-bias lateral bypass manoeuvres; indicates the robot misjudging available corridor width on the left.

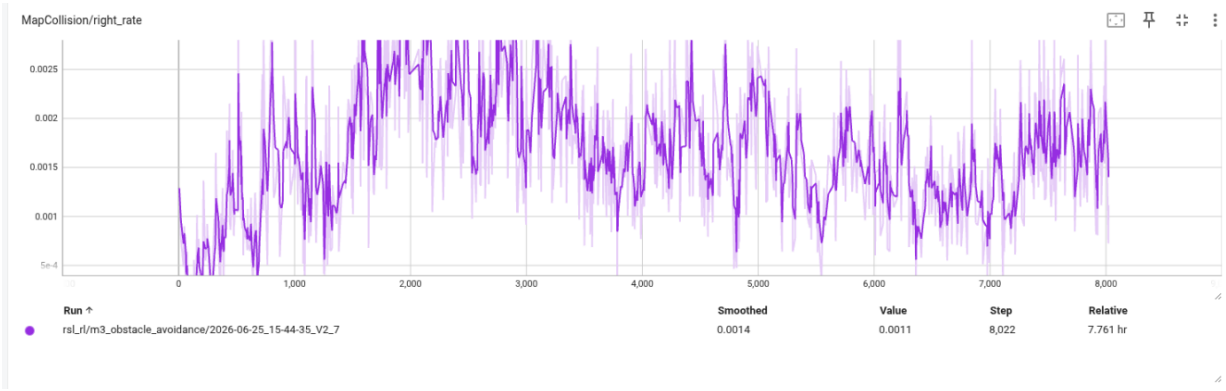

Figure 26. Static collision rate from the right side. Relevant to right-bias bypass manoeuvres.

15. Results

Controlled benchmark experiments comparing the RL policy against DWB, MPPI, and Regulated Pure Pursuit are pending completion of the training phase. The results section will be populated with quantitative metrics (success rate, time to goal, collision count, number of stops, minimum dynamic obstacle clearance) and trajectory visualisations following the benchmark runs.

Table 15. Benchmark results table (to be completed after controlled experiments).

Controller	Type	Success Rate	Avg. Time (s)	Collisions	Stops
DWB	Classical	—	—	—	—
MPPI	Classical	—	—	—	—
Regulated Pure Pursuit	Classical	—	—	—	—
PPO + ScanTransformer (this work)	Learned	—	—	—	—

16. Discussion

16.1 Policy Std Saturation and Curriculum Stagnation

The most significant actionable finding from run V2_7 is the policy standard deviation clamped at $\sigma = 1.0$ from step 600. RSL-RL enforces $\log\_std \in [-4, +1]$; without an entropy coefficient, the gradient for log_std comes only from the PPO surrogate loss, which does not penalise high entropy. The result is a policy that explores with maximal spread (within the clamp) indefinitely. The theoretical entropy of a 3D Gaussian with $\sigma = 1.0$ is $H = 4.257$ nats, which matches the observed flat entropy curve exactly.

High σ means a significant fraction of sampled actions are clipped by the physical velocity bounds before being applied, introducing trajectory noise that is uncorrelated with policy intent. This likely increases the variability of episode outcomes and prevents a clean 70 % success-rate threshold crossing. The fix is straightforward: lower the log-std upper clamp from +1 to +0 (or even −0.5), or add entropy_coeff = 0.005 to create gradient pressure toward smaller std.

16.2 Reward Structure Observations

The corridor-aware bypass rewards (`wall_aware_dynamic_bypass`, `dynamic_yield_no_side_space`, `dynamic_forward_blocked`) condition bypass direction on available lateral corridor width. This is a novel design intended to prevent the policy from selecting bypass directions that lead to wall contact. The directional collision analysis in Section 14.6 provides empirical feedback on whether this is working — high left/right collision rates would indicate that the corridor-width conditioning is not providing sufficient gradient signal.

16.3 Architecture Suitability

The ScanHistoryTransformerActor processes all 144 rays simultaneously via global self-attention, enabling each ray to relate to all others within a single forward pass. This is not possible with MLP-based scan encoders that treat rays independently or with 1D convolutions that have limited receptive fields. The three-way pooling strategy provides complementary representations: min-pool captures the closest obstacle; attention-pool focuses on task-relevant rays; mean-pool provides global occupancy density. Together they give the actor a richer view of the environment than any single aggregation method alone.

One architectural consideration is the fixed 144-ray input dimension. If the physical sensor has a different ray count, the `pos_embed` layer requires reinitialisation. The `lidar_rays` domain randomisation stage partially mitigates this by simulating dropout, but does not change the sequence length.

17. Conclusion

NavRL Bench presents a complete pipeline for training, validating, and deploying a reinforcement-learning local controller for the ROSMASTER M3 mecanum-wheel robot. The ScanHistoryTransformerActor encodes 8-frame lidar history across 144 rays into a 20-feature per-ray representation and processes it through a 3-layer transformer encoder with three-way pooling, enabling implicit obstacle motion inference without explicit dynamic state.

Training run V2_7 demonstrates sustained mean reward improvement over 8,022 steps and 7.76 hours. The primary blocking issue is policy standard deviation saturation at $\sigma = 1.0$ (the RSL-RL log-std upper clamp) from step 600, which prevents action precision from improving and holds the success rate below the 70 % curriculum promotion threshold. Resolving this via a tighter clamp or a small entropy coefficient is the highest-priority next training configuration change.

Following training completion, the benchmark comparison against DWB, MPPI, and Regulated Pure Pursuit will quantify whether the learned policy's mecanum lateral bypass capability provides a measurable advantage in dynamic-obstacle scenarios over classical Nav2 controllers. Results will be reported in the final version of this document.